UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL

CARRERA DE SOFTWARE

ALGORITMOS Y LÓGICA DE PROGRAMACIÓN

GUÍA APE 5 – PRIMERA PARTE

Manejo de ficheros en Java: crear, escribir y eliminar archivos

Estudiante: Paredes Acosta Dereck Shair

Docente: Ing. José Caiza

Semestre: Primer semestre

Periodo académico: Julio – Diciembre 2026

Repositorio GitHub: https://github.com/dereckflux/T2.4Fase1APE5.-Creaci-n-Lectura-y-Escritura-de-Ficheros

Ambato – Ecuador

Noviembre 2026

Ejercicios APE 5 – Manejo de ficheros en Java

Estudiante: Fher Dorian Calderon Carvajal

Universidad Técnica de Ambato – Carrera de Software

Ejercicio 1. Crear un fichero en Java

Enunciado:

Desarrollar un programa en Java que permita crear un archivo de texto llamado curso.txt.

Explicación de la solución:

Se crea un objeto File con el nombre curso.txt y se llama a createNewFile(), que devuelve true si el archivo se creó y false si ya existía. Con getName() se muestra el nombre del archivo, y el bloque try-catch controla la IOException que puede ocurrir si no hay permisos o la ruta no es válida.

Código fuente (Ejercicio1CrearFichero.java):

```
import java.io.File;
import java.io.IOException;

public class Ejercicio1CrearFichero {

    public static void main(String[] args) {
        // Se crea el objeto File con el nombre del archivo
        File archivo = new File("curso.txt");

        try {
            // createNewFile() devuelve true si lo crea y false si ya existía
            if (archivo.createNewFile()) {
                System.out.println("El archivo se creó correctamente.");
            } else {
                System.out.println("El archivo ya existe.");
            }
            System.out.println("Nombre del archivo: " + archivo.getName());

        } catch (IOException e) {
            System.out.println("Ocurrió un error al crear el archivo: " + e.getMessage());
        }
    }
}
```

Resultado de la ejecución:

```
--- Primera ejecución ---
El archivo se creó correctamente.
Nombre del archivo: curso.txt

--- Segunda ejecución ---
El archivo ya existe.
Nombre del archivo: curso.txt
```

Ejercicio 2. Escribir información en un fichero

Enunciado:

Desarrollar un programa en Java que permita ingresar un mensaje por teclado y almacenarlo en un archivo de texto existente llamado estudiantes.txt.

Explicación de la solución:

Con Scanner se lee el mensaje que escribe el usuario. Luego se abre estudiantes.txt con FileWriter usando el parámetro true, para que el texto nuevo se agregue al final sin borrar lo que ya tenía. El mensaje se guarda con write(), el archivo se cierra con close() y cualquier error de escritura se controla con try-catch.

Código fuente (Ejercicio2EscribirFichero.java):

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio2EscribirFichero {

    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Escriba un mensaje: ");
        String mensaje = teclado.nextLine();

        try {
            // El parámetro true hace que se agregue al final sin borrar lo anterior
            FileWriter escritor = new FileWriter("estudiantes.txt", true);
            escritor.write(mensaje + System.lineSeparator());
            escritor.close();

            System.out.println("La información se guardó correctamente en
estudiantes.txt");

        } catch (IOException e) {
            System.out.println("Ocurrió un error al escribir en el archivo: " +
e.getMessage());
        }

        teclado.close();
    }
}
```

Resultado de la ejecución:

```
Escriba un mensaje: Hola, soy Dorian
La información se guardó correctamente en estudiantes.txt

Escriba un mensaje: Segundo mensaje
La información se guardó correctamente en estudiantes.txt

Contenido de estudiantes.txt:
Hola, soy Dorian
Segundo mensaje
```

Ejercicio 3. Eliminar un fichero en Java

Enunciado:

Desarrollar un programa en Java que permita eliminar el archivo de texto estudiantes.txt.

Explicación de la solución:

Se crea un objeto File con el nombre estudiantes.txt y se llama a delete(), que devuelve true si el archivo se eliminó. Con un if-else se muestra un mensaje de confirmación o, en caso contrario, un mensaje indicando que no se pudo eliminar (por ejemplo, porque el archivo no existe o está abierto).

Código fuente (Ejercicio3EliminarFichero.java):

```
import java.io.File;

public class Ejercicio3EliminarFichero {

    public static void main(String[] args) {
        // Se indica el archivo que se desea eliminar
        File archivo = new File("estudiantes.txt");

        // delete() devuelve true si el archivo se eliminó
        if (archivo.delete()) {
            System.out.println("El archivo " + archivo.getName() + " se eliminó
correctamente.");
        } else {
            System.out.println("No se pudo eliminar el archivo " + archivo.getName()
                    + ". Verifique que exista y que no esté abierto por otro programa.");
        }
    }
}
```

Resultado de la ejecución:

```
--- Primera ejecución ---
El archivo estudiantes.txt se eliminó correctamente.

--- Segunda ejecución ---
No se pudo eliminar el archivo estudiantes.txt. Verifique que exista y que no esté abierto
por otro programa.
```

Conclusiones

1. La clase File permite representar un archivo y realizar operaciones básicas sobre él, como crearlo con createNewFile(), obtener su nombre con getName() o eliminarlo con delete().
2. FileWriter con el parámetro true permite agregar información al final de un archivo sin perder el contenido anterior, y es importante cerrarlo con close() para que los datos se guarden.
3. El uso de try-catch con IOException evita que el programa se detenga de forma inesperada cuando ocurre un error al trabajar con archivos.